§6.E Capability-Honesty Audit — magnet & download-label (2026-06-08)

Scope: three §6.E (Capability Honesty) questions across `core/tracker/{rutracker,rutor,gutenberg}`. Method: read descriptor capabilities, the feature impls they map to, the use-cases/parsers behind them, and the tests that pin the contract. Vocabulary follows §11.4.6 — claims are stated as fact with `file:line` evidence, or marked `UNCONFIRMED:`.

All `file:line` references are against the working tree at audit time.

W4a — RuTracker sync vs async magnet → BLUFF (CONFIRMED)

Declared capability

`RuTrackerDescriptor.capabilities` declares `TrackerCapability.MAGNET_LINK`:

- `core/tracker/rutracker/src/main/kotlin/lava/tracker/rutracker/RuTrackerDescriptor.kt:36`

`RuTrackerClient.getFeature(DownloadableTracker::class)` resolves to a non-null `RuTrackerDownload` (gated on `TORRENT_DOWNLOAD`, which is also declared):

- `core/tracker/rutracker/src/main/kotlin/lava/tracker/rutracker/RuTrackerClient.kt:60`

So `MAGNET_LINK` is reachable through `DownloadableTracker.getMagnetLink`.

Actual code behavior

`DownloadableTracker.getMagnetLink` is documented as a **synchronous** surface that returns a magnet when one is available without an HTTP fetch, else null:

- `core/tracker/api/src/main/kotlin/lava/tracker/api/feature/DownloadableTracker.kt:8-9`

`RuTrackerDownload.getMagnetLink` delegates to `GetMagnetLinkUseCase`:

- `core/tracker/rutracker/src/main/kotlin/lava/tracker/rutracker/feature/RuTrackerDownload.kt:28`

`GetMagnetLinkUseCase.invoke` **unconditionally returns null** — it is a stub:

- `core/tracker/rutracker/src/main/kotlin/lava/tracker/rutracker/domain/GetMagnetLinkUseCase.kt:13` → `operator fun invoke(id: String): String? = null`

The production wiring uses exactly this stub (no cache, no alternate path):

- `core/tracker/rutracker/src/main/kotlin/lava/tracker/rutracker/RuTrackerSubgraphBuilder.kt:94` → `val getMagnetLink = GetMagnetLinkUseCase()`
- `core/tracker/rutracker/src/main/kotlin/lava/tracker/rutracker/RuTrackerSubgraphBuilder.kt:111` → `val download = RuTrackerDownload(getTorrentFile, getMagnetLink, tokenProvider)`

Therefore for EVERY id, on the production stack, `getMagnetLink` returns null.

Why this is a bluff (and NOT merely an "honest synchronous absence")

RuTracker DOES carry the magnet — the production mappers populate `TorrentItem.magnetUri` from the parsed `magnetLink` for both topic and search surfaces:

- topic: `core/tracker/rutracker/src/main/kotlin/lava/tracker/rutracker/mapper/TopicMapper.kt:121` → `magnetUri = data?.magnetLink`
- search/torrent DTO: `core/tracker/rutracker/src/main/kotlin/lava/tracker/rutracker/mapper/RuTrackerDtoMappers.kt:305` and `:155`
- the parsers extract it: `domain/ParseTorrentUseCase.kt:22`, `domain/ParseTopicPageUseCase.kt:56`

This is the **identical situation** RuTor was in before its W4b fix: the magnet is genuinely parsed and surfaced through topic/search, but the synchronous `getMagnetLink` ignores it and returns a hardcoded null. RuTor closed the gap with a process-lifetime `RuTorMagnetCache` populated on topic/search fetch (see W4b). RuTracker has NOT adopted that pattern — its `getMagnetLink` can never return non-null for any id under any sequence of user actions.

A capability that is declared but whose only reachable implementation is a hardcoded `null` is the canonical §6.E bluff: "capability declared ⇒ feature interface returned ⇒ at least one real-stack test exists for the capability" (root CLAUDE.md §6.E). There is no real-stack test in which RuTracker's `getMagnetLink` returns a real magnet, because it cannot.

Verdict: BLUFF. `MAGNET_LINK` is declared but `getMagnetLink` returns `null` for all ids.

Recommended action (orchestrator — production-code fix required)

Mirror the RuTor fix exactly:

1. Add a `RuTrackerMagnetCache` (or reuse a shared `MagnetCache`) — process-lifetime `ConcurrentHashMap<String,String>` keyed by rutracker `torrentId`.
2. Populate it from the real parse path: in `RuTrackerTopic.getTopic` / `RuTrackerSearch.search`, call `cache.put(item.torrentId, item.magnetUri)` after mapping (the magnet is already present at `TopicMapper.kt:121` / `SearchPageMapper.kt:70`).
3. Replace the `GetMagnetLinkUseCase` stub so `RuTrackerDownload.getMagnetLink` reads the cache (honest null only when no fetch has surfaced the id).
4. Wire the shared cache instance through `RuTrackerSubgraphBuilder` (singleton-equivalent, one instance shared by topic/search/download — same as RuTor's clone path).

Alternative (if the project decides RuTracker genuinely cannot expose a sync magnet): DROP `TrackerCapability.MAGNET_LINK` from `RuTrackerDescriptor.capabilities` (the LF-5 precedent at `RuTrackerDescriptor.kt:18-27` already did this for UPLOAD/ USER_PROFILE). The cache fix is preferred because the data demonstrably exists.

Falsifiable test written (FAILS now, PASSES after the cache fix)

`core/tracker/rutracker/src/test/kotlin/lava/tracker/rutracker/feature/RuTrackerMagnetExposureTest.kt`

W4b — RuTor sync magnet → HONEST (CONFIRMED)

Declared capability

`RuTorDescriptor.capabilities` declares `TrackerCapability.MAGNET_LINK`:

- `core/tracker/rutor/src/main/kotlin/lava/tracker/rutor/RuTorDescriptor.kt:37`

Actual code behavior — the sync path genuinely returns a parsed magnet

`RuTorDownload.getMagnetLink` reads from `RuTorMagnetCache`:

- `core/tracker/rutor/src/main/kotlin/lava/tracker/rutor/feature/RuTorDownload.kt:43` → `override fun getMagnetLink(id: String): String? = magnetCache.get(id)`

The cache is populated by the REAL production topic/search fetch paths from genuinely-parsed `magnetUri`:

- topic: `core/tracker/rutor/src/main/kotlin/lava/tracker/rutor/feature/RuTorTopic.kt:64` → `magnetCache.put(id, baseDetail.torrent.magnetUri)`
- search: `core/tracker/rutor/src/main/kotlin/lava/tracker/rutor/feature/RuTorSearch.kt:65` → `result.items.forEach { magnetCache.put(it.torrentId, it.magnetUri) }`
- cache impl: `core/tracker/rutor/src/main/kotlin/lava/tracker/rutor/magnet/RuTorMagnetCache.kt:37-43` (`put` ignores blanks; `get` returns the stored magnet or null)

So after a user opens a topic OR runs a search that surfaces an id, `getMagnetLink(id)` returns the exact parsed `magnet:?xt=urn:btih:...` string — not a fabricated value, and an honest `null` only when no fetch has surfaced that id (matching the `DownloadableTracker` contract at `DownloadableTracker.kt:8`).

Test that proves it

`core/tracker/rutor/src/test/kotlin/lava/tracker/rutor/feature/RuTorMagnetExposureTest.kt`

- `getMagnetLink exposes the parsed magnet after the topic page surfaces it` (lines 57-94): drives real `RuTorTopic.getTopic` → asserts `download.getMagnetLink` equals `detail.torrent.magnetUri` and starts with `magnet:?xt=urn:btih:`.
- `getMagnetLink exposes the parsed magnet after a search row surfaces it` (lines 96-124): same via real `RuTorSearch.search`.
- `getMagnetLink returns null for an id never surfaced (honest absence)` (lines 126-134).
- Carries a documented FALSIFIABILITY REHEARSAL (lines 36-39): reverting `RuTorDownload.getMagnetLink` to `return null` fails the first two blocks.

Verdict: HONEST. Declaration matches reality; a real-stack test pins it.

Recommended action

None. The honest contract is already pinned by `RuTorMagnetExposureTest`. No production fix needed. (No new regression test written — the existing one is sufficient and already covers the sync magnet contract end-to-end.)

W3 — Gutenberg download label → HONEST (CONFIRMED)

Declared capability

`GutenbergDescriptor.capabilities` declares ONLY `SEARCH`, `BROWSE`, `TOPIC`:

- `core/tracker/gutenberg/src/main/kotlin/lava/tracker/gutenberg/GutenbergDescriptor.kt:32-38`
- `TORRENT_DOWNLOAD` and `MAGNET_LINK` are intentionally ABSENT, documented at `GutenbergDescriptor.kt:15-25` and `:36-37`.

Actual code behavior matches the reality (HTTP e-books, not .torrent)

- `GutenbergDownload.downloadTorrentFile` fetches book metadata and downloads the best EPUB/text/HTML format over HTTP — it does NOT produce a `.torrent`: `core/tracker/gutenberg/src/main/kotlin/lava/tracker/gutenberg/feature/GutenbergDownload.kt:29-39`
- `GutenbergDownload.getMagnetLink` returns null (Gutenberg has no magnets): `GutenbergDownload.kt:41`
- Because `TORRENT_DOWNLOAD` is undeclared, `GutenbergClient.getFeature(DownloadableTracker::class)` returns null — the HTTP-download impl is wired but NOT exposed through the torrent-download surface. This is the documented "mirroring the Internet Archive provider" posture (`GutenbergDescriptor.kt:22-24`).

Test that proves it

`core/tracker/gutenberg/src/test/kotlin/lava/tracker/gutenberg/GutenbergCapabilityHonestyTest.kt`

- `every declared capability resolves to a non-null feature` (lines 65-88).
- `download capability is honest about the artifact it produces` (lines 90-143): asserts that because `TORRENT_DOWNLOAD` is NOT declared, `getFeature(DownloadableTracker)` MUST be null — so a consumer never receives a non-`.torrent` artifact through the torrent-download surface (lines 128-142). The historical-bluff path (declare TORRENT_DOWNLOAD, return EPUB bytes) is asserted against by the bencode `'d'` first-byte check (lines 122-127).

Verdict: HONEST. The label/capability matches reality (no `.torrent` claimed); pinned by

`GutenbergCapabilityHonestyTest`.

Recommended action

None. The TORRENT_DOWNLOAD→correct-label fix has landed and is pinned by the existing honesty test. No production fix needed. (No new regression test written — existing coverage is sufficient.)

Summary

Q	Provider	Declared	Reality	Verdict	Production fix needed
W4a	RuTracker	`MAGNET_LINK`	`getMagnetLink` returns `null` for all ids (stub), despite magnet being parsed & present in mappers	**BLUFF**	**YES** — adopt the RuTor cache pattern
W4b	RuTor	`MAGNET_LINK`	`getMagnetLink` returns the genuinely-parsed magnet via cache populated on topic/search fetch	**HONEST**	No
W3	Gutenberg	`SEARCH/BROWSE/TOPIC` (no `TORRENT_DOWNLOAD/MAGNET_LINK`)	HTTP e-books, download impl wired but unexposed via getFeature	**HONEST**	No

Only **W4a (RuTracker MAGNET_LINK)** requires a production-code fix by the orchestrator. The falsifiable test for it is written and will FAIL against the current stub and PASS once the cache fix lands.